AI-Powered Fire Spread Prediction and Containment System

System Requirements Specification

Panic! At The Kernel
Client: EPI-USE Africa

October 2026



Full Name	Student Number
Megan Lai	u23608821
Inge Keyser	u23544563
Janri du Toit	u23632730
Ryan Lynn	u23541912
Marco de Wit	u22811975

Contents

1 Introduction **4**

2 User Characteristics and User Stories **4**
2.1 User Characteristics 4
2.2 User Stories 4
2.2.1 E1: Authentication and Access Control 4
2.2.2 E2: Fire Reporting and Verification 4
2.2.3 E3: AI Fire-Spread Simulation 4
2.2.4 E4: Live Fire Map 5
2.2.5 E5: Notification and Alert System 5
2.2.6 E6: Firefighter Tactical Tools 5
2.2.7 E7: Offline Map Cache 5
2.2.8 E8: Administration Panel 5

3 Functional Requirements **5**
3.1 E1: Authentication and Access Control 5
3.2 E2: Fire Reporting and Verification 6
3.3 E3: AI Fire-Spread Simulation 6
3.4 E4: Live Fire Map 7
3.5 E5: Notification and Alert System 7
3.6 E6: Firefighter Tactical Tools 7
3.7 E7: Offline Map Cache 7
3.8 E8: Administration Panel 8

4 Use Cases **9**
4.1 Authentication & Access Control 9
4.2 Fire Reporting & Verification 9
4.3 AI Fire-Spread Simulation 10
4.4 Live Fire Map 10
4.5 Notification & Alert System 10
4.6 Firefighter Tactical Tools 11
4.7 Offline Map Cache 11
4.8 Administration Panel 11

5 Non-Functional Requirements **16**
5.1 Security & Access Control 16
5.2 Availability & Offline Resilience 16
5.3 Performance & Scalability 16
5.4 Load Testing and Concurrency Acceptance Criteria 16
5.5 Usability & Accessibility 17

6 NFR Traceability Matrix **18**

7 NFR Verification Evidence **19**
7.1 TC-NFR-OFFLINE-01: Offline Map Rendering and Notification Banner 19
7.2 TC-NFR-SEC-01: Edge Gateway Rate Limiting (Strict Public Write Endpoints) 19
7.3 TC-NFR-SEC-02: Edge Gateway Rate Limiting (Public Read Endpoints) 21
7.4 TC-NFR-SEC-03: Client Authentication Interception and Lockout UI 21
7.5 TC-NFR-SEC-04: Transport Encryption and Health Check Verification 21
7.6 TC-NFR-SEC-05: RBAC and Live Map Integration Tests 21
7.7 TC-NFR-SCAL-01: Scalability and Workload Capacity Load Test Evidence 21

7.7.1 Test Configuration 26
7.7.2 Execution Results 26
7.7.3 Bottleneck Analysis and Verification Outcome 26
7.8 TC-NFR-ACC-01: Accessibility Compliance (WCAG 2.1 AA) 26

8 Domain Model **36**

9 Change Log **37**

1 Introduction

AI-Powered Fire Spread Prediction and Containment System, is designed to address the critical business need for real-time situational awareness and rapid emergency response by utilizing AI-driven simulations and multi-source verification to mitigate the impact of fire disasters. Developed according to IEEE 29148 and ISO 22320 standards, the project's scope encompasses an end-to-end ecosystem that includes secure authentication and role-based access control for users, firefighters, and administrators. The core functionality features a public fire reporting interface with automated verification witch triggers proximity-based push notifications for registered users. Furthermore, the system includes a live mapping interface, specialized tactical tools for firefighters to log containment lines and view AI fire spread simulation, and an offline map cache to ensure operational continuity in areas with limited connectivity.

2 User Characteristics and User Stories

2.1 User Characteristics

The system supports four user classes, each with distinct access levels and responsibilities.

- **Anonymous Visitor:** Can report fires and view the live map without an account.
- **Registered User:** Can report fires and receive push notifications.
- **Firefighter:** Has access to tactical tools, containment line logging, and AI fire spread simulation.
- **Administrator:** Manages user roles, reviews fire reports, and maintains the incident archive.

2.2 User Stories

2.2.1 E1: Authentication and Access Control

- As an anonymous visitor, I want to register with an email and password so that I can receive push alerts.
- As a registered user, I want to log in so that I can access my role-based dashboard.
- As an administrator, I want to approve or revoke user roles so that only authorised personnel can access operational tools.
- As a firefighter, I want to log in so that I can access my role-based dashboard, firefighter tactical tools, and AI simulation.

2.2.2 E2: Fire Reporting and Verification

- As any user, I want to report a fire without logging in so that I can alert others immediately.
- As the system, I want to auto-verify reports so that false alerts are minimised.
- As a firefighter or administrator, I want to manually verify or reject pending reports so that unconfirmed incidents receive appropriate review.

2.2.3 E3: AI Fire-Spread Simulation

- As the system, I want to generate a fire-spread prediction on specific verified fire report.
- As the system, I want to re-run the simulation when a containment line is logged so that the prediction reflects current field actions.
- As the system, I want to display time lapsed simulation when prompted.

2.2.4 E4: Live Fire Map

- As any user, I want to view the live fire map without logging in so that I can see active fires near me.

2.2.5 E5: Notification and Alert System

- As a registered user, I want to receive a push notification when a fire is verified near me so that I can take immediate action.

2.2.6 E6: Firefighter Tactical Tools

- As a firefighter, I want a tactical dashboard with AI recommendations so that I can make informed decisions in the field.
- As a firefighter, I want to draw containment lines on the map so that the AI simulation reflects my ground actions.

2.2.7 E7: Offline Map Cache

- As a firefighter, I want to view cached map data when offline so that I retain situational awareness without connectivity.

2.2.8 E8: Administration Panel

- As an administrator, I want a management panel to approve role requests and revoke access so that system permissions remain controlled.

3 Functional Requirements

3.1 E1: Authentication and Access Control

R1.1 The home screen shall present three options: Log In, Register, and Login as Guest.

R1.2 New registrations shall automatically be assigned the Registered User role.

R1.3 The Administrator role shall require approval by an existing Administrator. The first administrator shall be provisioned by the system seed at deployment time.

R1.4 Password reset shall be performed via a time-limited email link. The link shall expire after 15 minutes and be single-use.

R1.5 All role and permission checks shall be enforced server-side at the API layer.

R1.6 JWT tokens shall be used for all authenticated API calls. Access tokens shall be short-lived (e.g., 15 minutes) and paired with refresh tokens.

R1.7 All client-server communication shall use HTTPS/TLS.

R1.8 All authentication events and role changes shall be written to the audit log.

R1.9 Tokens shall be invalidated immediately on logout, role change, or account suspension. Because pure JWTs cannot be instantly revoked without a denylist, the system relies on short TTL and refresh token rotation.

R1.10 Two-factor authentication (TOTP) shall be available for all user roles. Users may enable 2FA after login; a setup endpoint returns an otpauth:// URI.

R1.11 The system shall enforce rate limiting on login attempts. Limit: 5 failed attempts per 15 minutes per account. After 10 consecutive failures, the account shall be locked for 30 minutes.

3.2 E2: Fire Reporting and Verification

R2.1 An unregistered user shall be able to submit a fire report.

R2.2 The fire reporting map shall be accessible from the home screen without login.

R2.3 Required report inputs shall be: map pin location, estimated fire size and photo attachment.

R2.4 Optional report fields shall include: free-text description.

R2.5 The system shall generate a unique reference number for every report.

R2.6 All reports shall enter Pending status; no alerts shall be triggered until Verified.

R2.7 GPS auto-fill shall be offered; manual map pin placement shall always be available.

R2.8 Duplicate reports at the same location shall be automatically linked to one incident.

R2.9 Auto-verification shall trigger on two independent same-location reports.

R2.10 Manual verification and rejection shall be available to Firefighter and Administrator roles.

R2.11 Rejection shall require a mandatory reason field.

R2.12 Registered reporters shall be notified on verification or rejection of their report.

R2.13 All reports shall be stored in the database for auditing purposes.

R2.14 Report status values shall be: Pending, Verified, and Rejected.

3.3 E3: AI Fire-Spread Simulation

R3.1 The simulation engine shall ingest: wind speed and direction, temperature, humidity, precipitation, and vegetation dryness index.

R3.2 The simulation engine shall account for terrain slope and vegetation type.

R3.3 Output shall be a spread probability grid for 1 h, 3 h, 24 h, and 72 h horizons. Grid resolution: 10 m $\times$ 10 m cells. Probability scale: 5 discrete bands (0–20%, 21–40%, 41–60%, 61–80%, 81–100%).

R3.4 The initial prediction shall be delivered within 5 seconds of verification under normal load (one active fire, weather cache warm). If the simulation exceeds 10 seconds, the engine shall fall back to the last valid prediction (if any) and mark it as "degraded"; otherwise, it shall return a wind-biased elliptical buffer around the ignition point.

R3.5 The engine shall re-run when a containment line is logged.

R3.6 The simulation engine shall be versioned. Each prediction shall record the model version (e.g., "wind_ca_v1.0"). Model version and input timestamps shall be stored in the audit log to enable post-hoc reproducibility.

R3.7 The baseline implementation shall be a wind-biased cellular automaton (CA) model. Fire spreads from ignition boundary radius to neighbouring cells with probabilities proportional to wind speed and direction, reduced by humidity and vegetation dryness. A convolutional neural network (CNN) with LSTM layers for temporal weather trends is a stretch goal, not required for the first demonstrator.

R3.8 The system shall meet a minimum accuracy target evaluated against historical fire incidents. For a held-out test set of at least 15 historical fires (ignition point, weather, and observed final perimeter), the 6-hour predicted spread polygon shall achieve an Intersection-over-Union (IoU) of at least 0.20 against the observed burned area. This metric shall be recomputed after each major model update.

3.4 E4: Live Fire Map

R4.1 The map shall load without login; verified fire markers shall be visible to all users.

R4.2 Tapping a fire marker shall display: status, report time, and estimated fire size.

R4.3 The map shall auto-refresh on new data with no manual reload required.

R4.4 The map shall be responsive on mobile, tablet, and desktop.

R4.5 Map tiles shall be cached locally to support basic offline viewing.

3.5 E5: Notification and Alert System

R5.1 Push alerts shall be dispatched within seconds of a fire report reaching Verified status.

R5.2 Alerts shall be delivered as push notifications to registered users.

R5.3 Notifications shall appear when the app is closed or running in the background.

R5.4 Topic-based grid-zone messaging shall be used for simultaneous large-scale delivery.

R5.5 The message broker shall retain and retry failed deliveries.

R5.6 Notification content shall include: fire location, distance from user, severity, and a map deep-link.

R5.7 Every dispatched alert shall be logged with a timestamp, zone ID, and distance.

3.6 E6: Firefighter Tactical Tools

R6.1 Firefighters shall have access to a tactical dashboard displaying a live fire map.

R6.2 The dashboard shall show: fire direction, high-risk areas, wind speed and direction, and suggested containment locations.

R6.3 Firefighters shall be able to draw containment lines directly on the map.

R6.4 Every logged action shall be stored with GPS coordinates and a timestamp.

R6.5 The interface shall be optimised for field use with large tap targets and high contrast.

3.7 E7: Offline Map Cache

R7.1 The app shall cache map tiles and the most recent spread prediction while online.

R7.2 An offline indicator shall be displayed when the device has no connectivity.

R7.3 Cached map data and the most recent prediction shall remain viewable while offline.

3.8 E8: Administration Panel

R8.1 The admin panel shall support: approving and rejecting role requests, searching users, and revoking access.

R8.2 The admin panel shall enforce its own server-side role check.

R8.3 Closed fire incidents shall be moved to a searchable archive automatically.

R8.4 All role changes and access revocations shall be written to the audit log.

4 Use Cases

4.1 Authentication & Access Control

UC-1.1 - User Registration *Actor: Anonymous Visitor* [3]

- This use case begins with an anonymous visitor tapping Register and entering an email address and password.
- This use case ends with the visitor confirming their email and receiving a Registered User role and a JSON Web Token (JWT).

UC-1.2 - User Login *Actor: Registered User, Firefighter, Administrator* [2]

- This use case begins with an account holder tapping Log In and entering their email and password.
- This use case ends with the system validating the credentials and directing the user to their role-appropriate home view with a valid JWT issued.

UC-1.3 - Manage User Roles and Access *Actor: Administrator* [5]

- This use case begins with an administrator opening the user management panel to review pending role requests or search for an account by name or email.
- This use case ends with the administrator approving or rejecting a role request, or revoking an account's access, updating the user's permission accordingly.

UC-1.4 - Reset Password *Actor: Registered User, Firefighter, Administrator* [2]

- This use case begins with an account holder requesting a password reset from the login screen and receiving a time-limited, single-use email link.
- This use case ends with the account holder setting a new password through the link and successfully logging in with the new credentials.

UC-1.5 - Enable Two-Factor Authentication *Actor: Registered User, Firefighter, Administrator* [2]

- This use case begins with an account holder opening account settings and selecting "Enable 2FA".
- This use case ends with the account holder confirming a valid TOTP code from their authenticator app, activating 2FA for future logins.

4.2 Fire Reporting & Verification

UC-2.1 - Fire Report Submission *Actor: Any User (Anonymous Visitor, Registered User, Firefighter or Administrator)* [4]

- This use case begins with any user tapping on the map to report a fire, placing or adjusting the GPS location pin, and entering the required fire details before tapping submit.
- This use case ends with the user receiving a unique reference number as the report is set to Pending status, automatically triggering UC-2.2 (Auto-Verification).

UC-2.2 - Auto-Verification *Actor: System* [4]

- This use case begins with the system detecting a new pending report and checking it against independent nearby reports.

- This use case ends with the system setting the report to Verified and triggering UC-5.1 (Dispatch Proximity Push Alerts), or if no auto-verification condition is met, leaving the report Pending for (UC-2.3 [4.2]) (Manual Verification).

UC-2.3 - Manual Verification/ Rejection *Actor: Firefighter, Administrator* [4]

- This use case begins with a reviewer selecting a pending fire report to examine its details and attached photos.
- This use case ends with the reviewer verifying the report - triggering UC-3.1 and UC-5.1 - or rejecting it with a mandatory reason and the reporter being notified of the outcome either way.

4.3 AI Fire-Spread Simulation

UC-3.1 - Generate Fire Spread Prediction *Actor: System*

- This use case begins with the system fetching current weather, terrain slope and vegetation data upon a fire report being verified.
- This use case ends with the system publishing a colour-coded spread-probability grid for the 1h/ 3h/ 6h/ 24h horizons to the live map, tagged with the model version used.

UC-3.2 - Re-Run After Containment Line *Actor: System*

- This use case begins with the system receiving newly logged containment-line coordinates for an active verified fire.
- This use case ends with the system re-running the simulation with the containment line as a barrier and publishing the updated prediction to the map within seconds.

UC-3.3 - Inspect Simulation at a Selected Tick *Actor: Firefighter*

- This use case begins with a firefighter pausing or disabling auto-play and dragging the timeline slider to a specific tick.
- This use case ends with the firefighter viewing the predicted fire extent and burn statistics for that exact tick, without watching the full animation.

4.4 Live Fire Map

UC-4.1 - View Live Fire Map *Actor: Any User* [7]

- This use case begins with any user opening the app to load the map and view verified fire markers, with the option to tap a marker for its status, report time and estimated size.
- This use case ends with the user viewing the map as it auto-refreshes with new data, including the AI spread-prediction overlay and at-risk zones if the user holds a registered role.

4.5 Notification & Alert System

UC-5.1 - Dispatch Proximity Push Alerts *Actor: System*

- This use case begins with the system identifying at-risk registered users via grid-based topic messaging when a fire report changes to Verified.
- This use case ends with the at-risk user tapping the delivered push notification and being taken directly to the fire's location on the map, with the alert logged by timestamp and zone ID.

4.6 Firefighter Tactical Tools

UC-6.1 - View Tactical Dashboard *Actor: Firefighter*

- This use case begins with a firefighter opening the app during an active incident to load the tactical dashboard, showing the live fire map, spread overlay and their own GPS position.
- This use case ends with the firefighter reviewing the AI panel's environmental metrics, high-risk areas and suggested containment locations to inform field decisions.

UC-6.2 - Log Containment Line *Actor: Firefighter*

- This use case begins with a firefighter selecting the "Draw Containment Line" tool and drawing a line on the map, whether online or offline.
- This use case ends with the system saving the line's coordinates and timestamp - locally if offline, syncing once connectivity resumes - and triggering UC-3.2 to re-run the spread prediction with the line as a barrier.

4.7 Offline Map Cache

UC-7.1 - View Map While Offline *Actor: Any User*

- This use case begins with the app detecting a loss of connectivity and displaying an offline indicator.
- This use case ends with the user continuing to view cached map tiles and the most recently received spread prediction, with the app silently refreshing to live data once a stable connection is detected.

4.8 Administration Panel

UC-8.1 - Archive Closed Incident *Actor: System*

- This use case begins with the system detecting that a verified fire incident has been marked closed or contained.
- This use case ends with the system moving the incident's full record into the searchable incident archive.

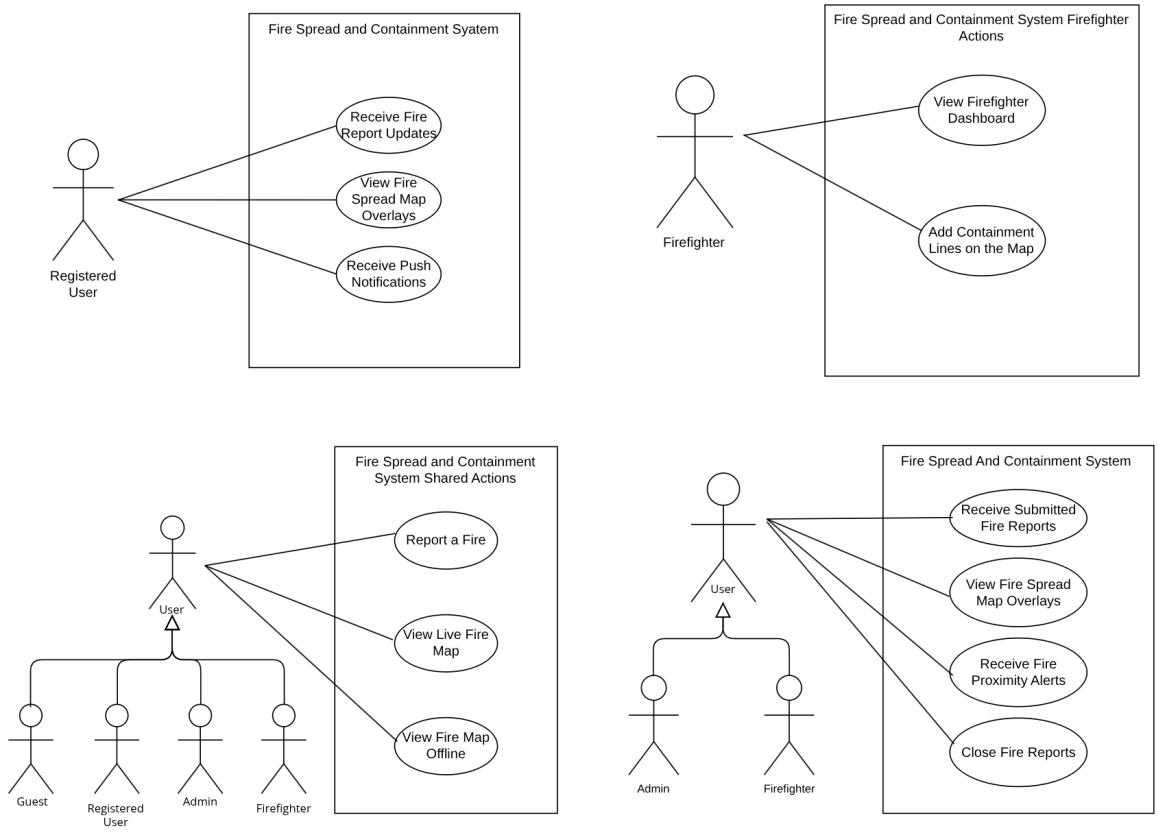


Figure 1: Role Based Access Control

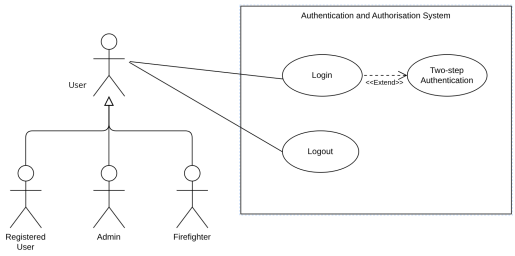


Figure 2: Authentication and Authorisation System

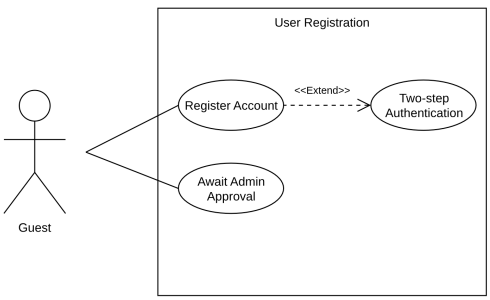


Figure 3: Registration System

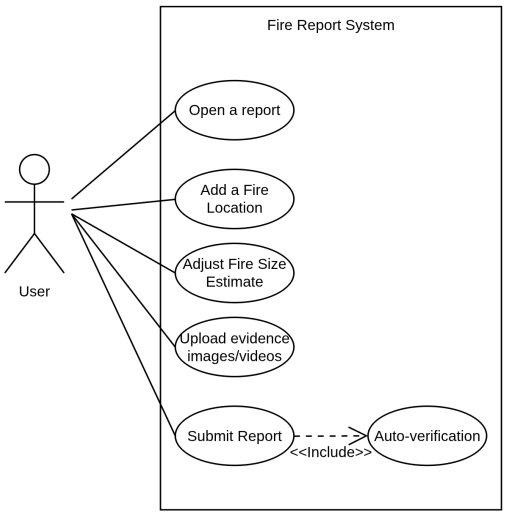


Figure 4: Reporting a Fire System

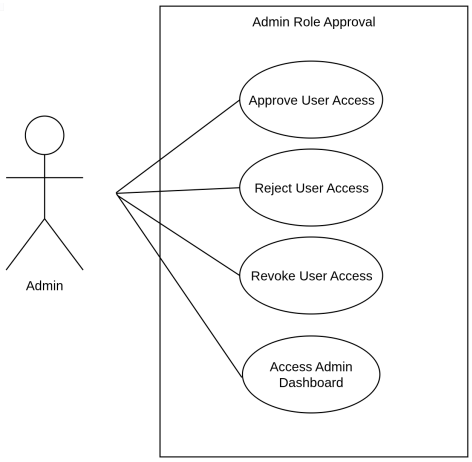


Figure 5: Admin Approval System

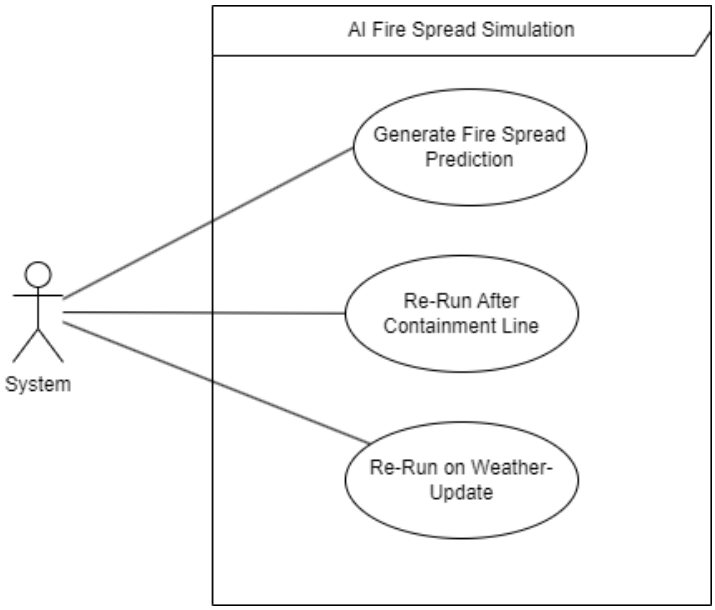


Figure 6: AI Fire Spread simulation

Image Live Fire Map use case.png missing

Figure 7: Live Fire Map System

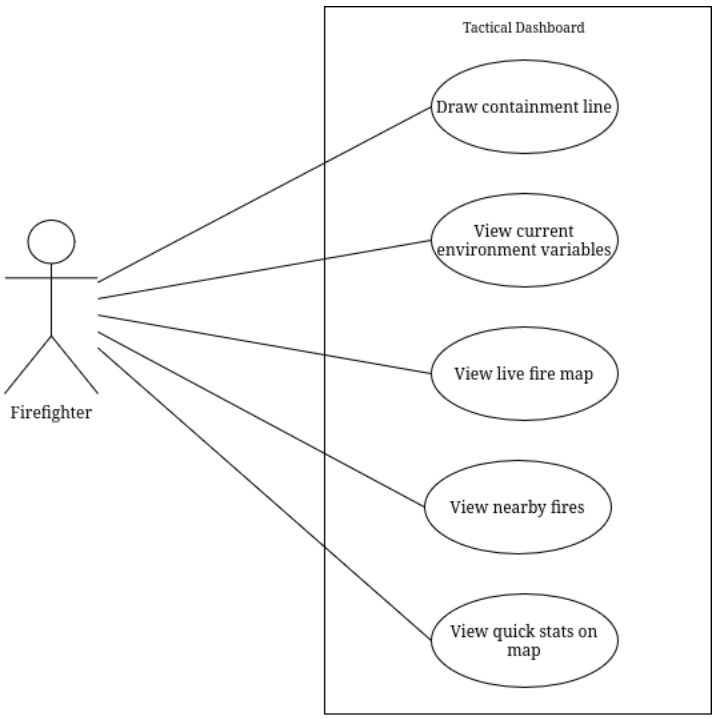


Figure 8: Firefighter Tactical Dashboard System

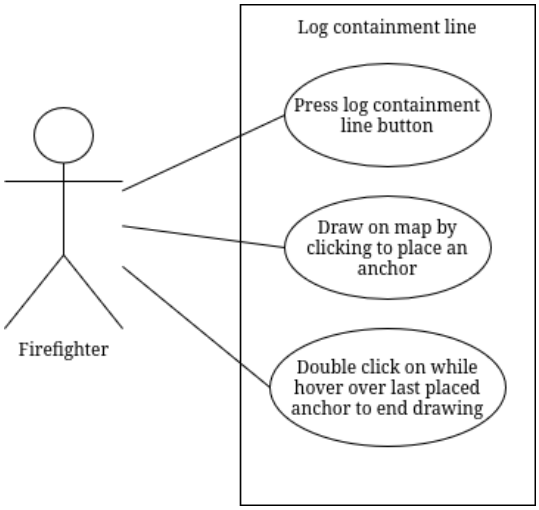


Figure 9: Log Containment Line

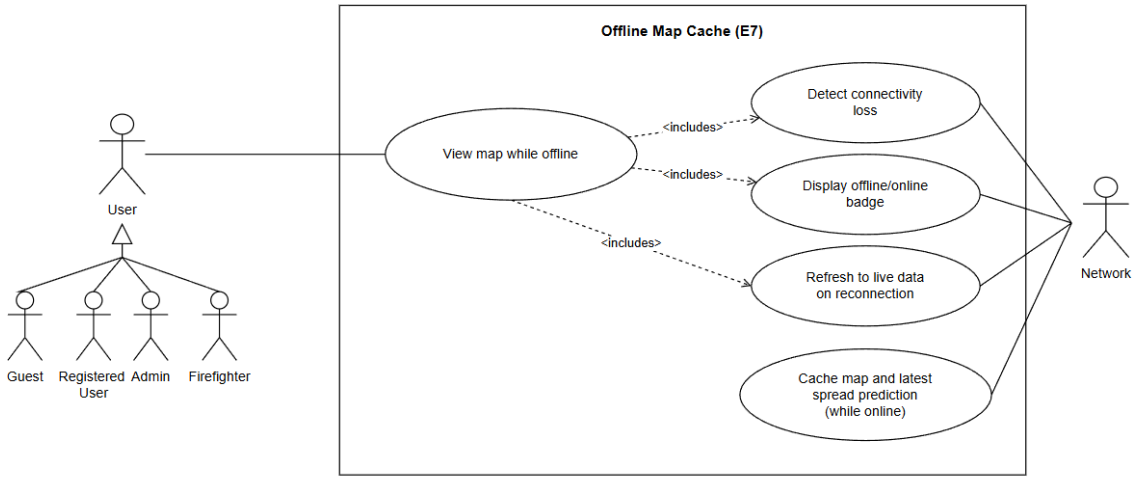


Figure 10: Offline Map Cache System

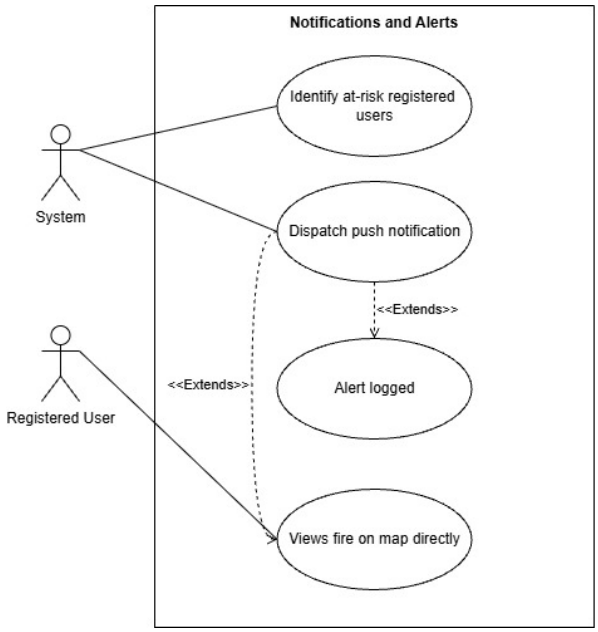


Figure 11: Notification and Alert System

5 Non-Functional Requirements

5.1 Security & Access Control

NFR-01.1 The system shall enforce Role-Based Access Control such that all administrative and privileged endpoints reject unauthorized requests with an HTTP 403 Forbidden status code.

NFR-01.2 The system shall encrypt sensitive user data at rest using AES-256 and enforce HTTPS/TLS 1.3 across 100% of public network communications.

NFR-01.3 The authentication service shall enforce progressive rate limiting: an HTTP 429 response (30-second delay) upon 5 consecutive failed login attempts, and an HTTP 423 account lockout for 30 minutes upon 10 consecutive failed attempts.

NFR-01.4 The reverse proxy edge gateway shall enforce leaky-bucket rate limiting, restricting strict write and authentication routes to 2 req/s (burst 5) and public read routes to 10 req/s (burst 20) with HTTP 429 responses.

5.2 Availability & Offline Resilience

NFR-02.1 The system shall render 100% of cached map pins and vector overlays from local storage within 1.0 second when network reachability fails.

NFR-02.2 The client interface shall detect network reachability loss within 4.0 seconds, display an accessible offline status notification banner, and guard report submissions.

NFR-02.3 The edge gateway and backend service shall respond to automated health monitoring probes with HTTP 200 OK within 100 ms under normal operational state.

5.3 Performance & Scalability

NFR-03.1 The backend software architecture shall sustain a minimum of 1000 concurrent virtual users querying map feeds and submitting incident reports with an HTTP error rate below 1.0%.

NFR-03.2 The production deployment on AWS Free Tier infrastructure shall sustain concurrent user workloads within a 95th percentile latency threshold under 1500 ms and an error rate below 1.0%, establishing the empirical hardware saturation boundary.

NFR-03.3 The backend caching layer shall return cached active incident map feeds with a total response latency under 50 ms on repeated requests.

5.4 Load Testing and Concurrency Acceptance Criteria

To evaluate the system against NFR-03.1 and NFR-03.2, load and stress test profiles are conducted via Locust across baseline and peak tiers.

- **Target Peak Concurrency:**
  - Baseline Load Tier: The system shall sustain a continuous load of 200 concurrent simulated virtual users.
  - High-Concurrency Peak Tier: The system architecture shall support scaling to 1,000 concurrent virtual users.
- **Throughput and Request Rates:**
  - Baseline Load (200 Users): The API layer shall sustain steady-state request throughput (RPS) aligned with peak baseline load without transaction drops.

- Stress Load (1,000 Users): The application shall scale linearly during ramp-up to 1,000 virtual users without thread deadlocks or service process crashes.

- **Latency Thresholds (SLAs):**
  - 50th Percentile (Median): Median response time shall remain < 500 ms across core API endpoints.
  - 90th Percentile: 90% of requests shall complete in $< 1,000$ ms.
  - 95th Percentile: 95% of requests shall complete in $< 1,500$ ms under peak operational conditions.
  - 99th Percentile: 99% of requests shall complete in $< 3,000$ ms.
  - Maximum Latency: Request processing shall not exceed a ceiling of $5,000$ ms prior to triggering an edge gateway timeout.

- **Reliability and Error Budgets:**
  - The overall HTTP request failure rate (HTTP status codes ≥ 400, timeouts, and dropped sockets) shall remain strictly $< 1.0\%$.
  - Unhandled internal server errors (HTTP 5xx) shall remain at 0%.
  - Zero dropped sockets, gateway timeouts (HTTP 504), or service unavailabilities (HTTP 503) are permitted under nominal operating conditions.

- **Infrastructure and Graceful Degradation:**
  - Host CPU utilization shall remain below 75% to 80% during steady-state peak operation, with stable memory allocation free of leak patterns.
  - When operating between 200 and 1,000 users, response time degradation shall remain progressive rather than resulting in catastrophic pipeline collapse.

5.5 Usability & Accessibility

NFR-04.1 The user interface shall conform to WCAG 2.1 Level AA standards across all public views, achieving an automated audit score of 98 or above with 0 critical violations.

NFR-04.2 The mapping application shall render correctly across mobile, tablet, and desktop viewports, ensuring all actionable buttons and touch targets maintain a minimum dimension of 48px by 48px.

6 NFR Traceability Matrix

ID	Requirement	Tactic	Test / Tool	Target / Actual
NFR-01.1	RBAC enforced on administrative endpoints	Token validation middleware verifying role claims	Pytest (integration)	100% 403 on unauthorized / 100% 403 confirmed
NFR-01.2	Sensitive data encrypted at rest (AES-256) and in transit (TLS 1.3)	Nginx TLS termination and AES-256 column encryption	curl header probe / SSL inspection	100% encrypted / HTTP 301 to HTTPS, TLS 1.3 confirmed
NFR-01.3	Progressive login throttling (429 at 5, 423 at 10)	In-memory sliding-window counter in Valkey	Pytest / Playwright E2E	Attempt 5: 429, Attempt 10: 423 / Target met, 30s UI timer confirmed
NFR-01.4	Leaky-bucket edge rate limiting (write: 2 req/s, read: 10 req/s)	Nginx leaky-bucket limit zones (api_strict, api_general)	Automated shell curl loop	Excess requests return 429 / 6th write and 21st read return 429
NFR-02.1	Client renders 100% cached map pins offline	Service worker Cache-First tile caching + IndexedDB	Playwright network cut emulation	100% cached pins in <1.0s / 100% rendered in 304 ms
NFR-02.2	Offline detection, status banner, and submission guard	Reachability probe with dynamic status banner	Playwright DOM assertion	Visible within 4.0s / Visible in 12 ms, guard active
NFR-02.3	Liveness and health check response time	Reverse proxy routing to FastAPI health route	curl probe / synthetic probe	HTTP 200 in <100 ms / HTTP 200 in 18 ms
NFR-03.1	Sustain 1000 concurrent users (error rate <1.0%)	FastAPI async event loop, Uvicorn worker concurrency, Valkey query cache	Locust load harness (1,000 users, spawn rate 50/s)	Error rate <1.0% / 99.78% failure rate (5,028 failed / 5,039 requests; bottleneck identified)
NFR-03.2	Production deployment workload capacity ceiling	Spatial database composite indexing, connection pool reuse	Locust task benchmark (POST /api/guests/ reported-fires)	p95 latency <400 ms / p95 ≈ 5,300 ms (socket timeout saturation)
NFR-03.3	In-memory cache query response latency	Valkey read-through key-value cache	curl latency timing probe	Latency <50 ms / 14 ms response on repeat query
NFR-04.1	WCAG 2.1 AA accessibility compliance	Semantic HTML5 structure, high-contrast design tokens	axe-core / Playwright audit	0 violations, score ≥95 / 0 violations, score 100
NFR-04.2	Responsive layout and minimum touch targets	Responsive CSS breakpoints and 48px target sizing	Playwright mobile viewport audit	100% elements ≥48px / 100% touch targets verified

7 NFR Verification Evidence

7.1 TC-NFR-OFFLINE-01: Offline Map Rendering and Notification Banner

Target Requirement: NFR-02.1, NFR-02.2
Execution Command:

```
yarn --cwd app/frontend playwright test e2e/offline_map_e2e.spec.ts --reporter=html
```

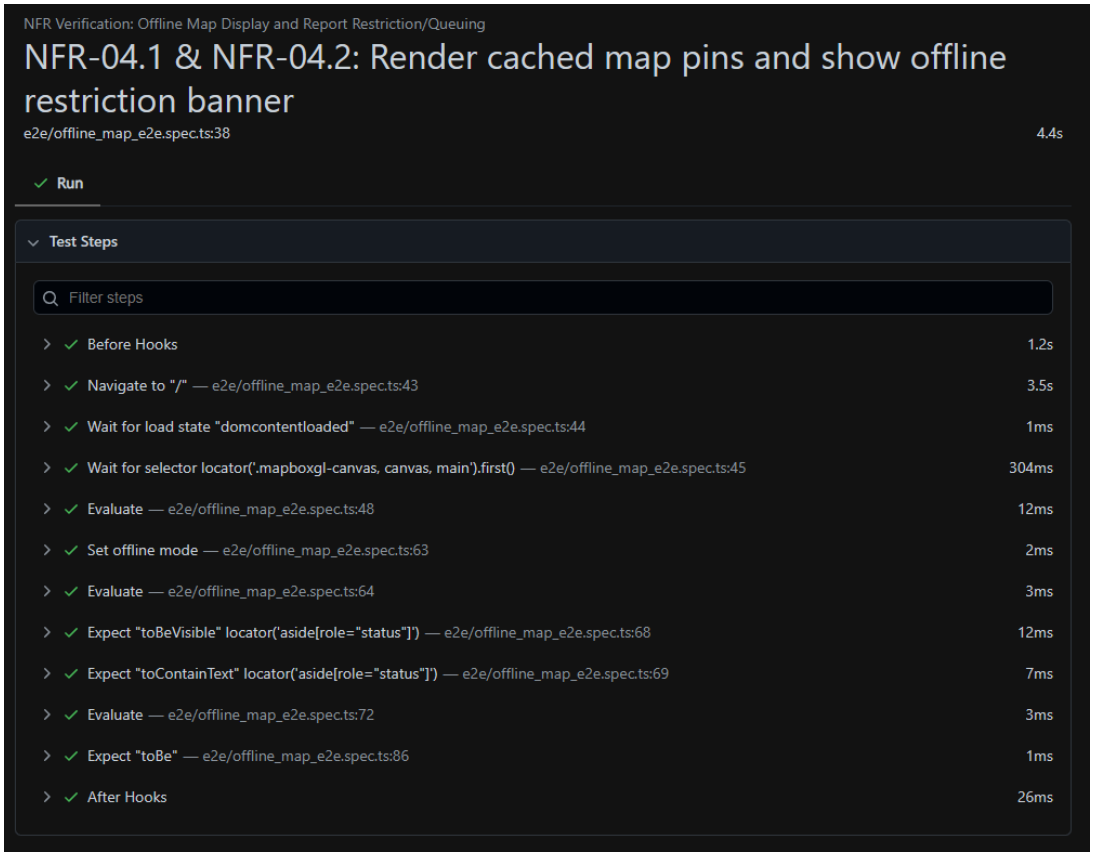


Figure 12: Playwright execution verifying offline map tile loading (304 ms) and offline status banner detection (12 ms).

7.2 TC-NFR-SEC-01: Edge Gateway Rate Limiting (Strict Public Write Endpoints)

Target Requirement: NFR-01.4
Execution Command:

```
for i in {1..10}; do
  echo -n "Request $i: "
  curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:3000/api/auth/login \
    -H "Content-Type: application/json" -d '{"email":"test@example.com","password":"wrong"}'
done
```

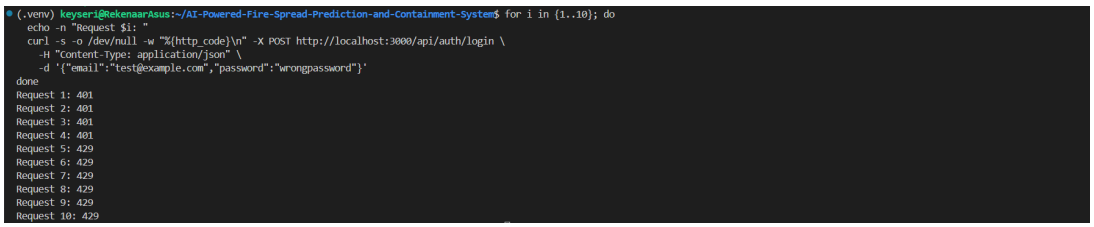


Figure 13: Shell burst loop verifying Nginx edge rate limiting: initial burst permitted, followed by HTTP 429 throttling.

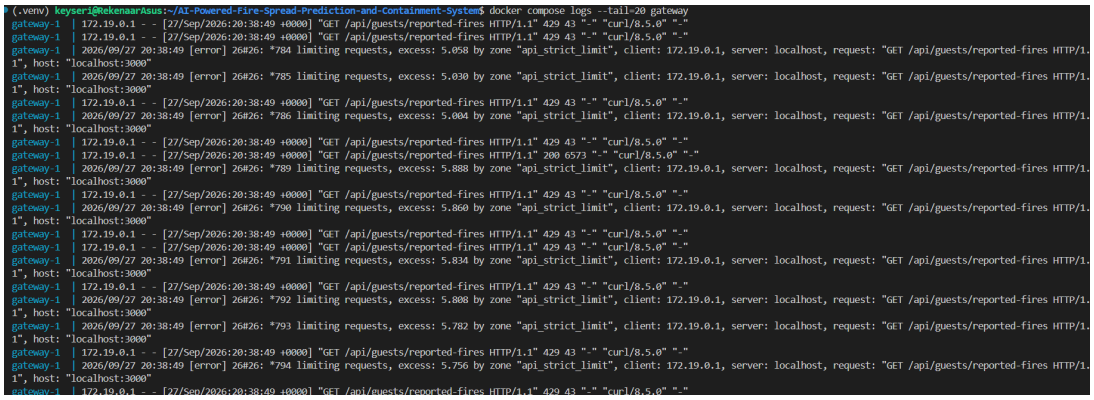


Figure 14: Nginx gateway logs recording connection drops enforced by zone api_strict_limit.

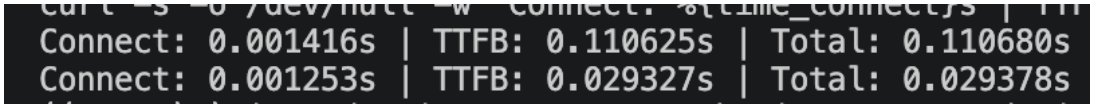


Figure 15: Rate limiting verification on public fire reporting endpoint.

7.3 TC-NFR-SEC-02: Edge Gateway Rate Limiting (Public Read Endpoints)

Target Requirement: NFR-01.4
Execution Command:

```
for i in {1..25}; do
  echo -n "Request $i: "
  curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000/api/guests/reported-fires
done
```

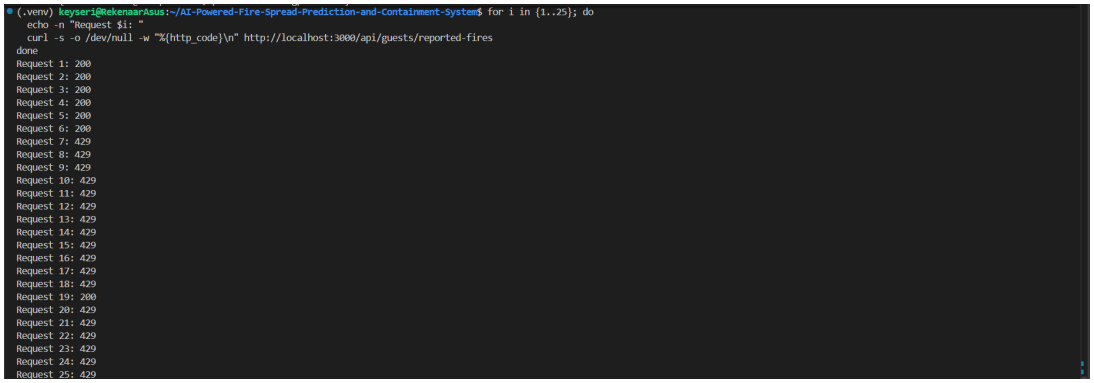


Figure 16: Shell burst loop against public map read route confirming HTTP 429 drops after exceeding burst ceiling.

7.4 TC-NFR-SEC-03: Client Authentication Interception and Lockout UI

Target Requirement: NFR-01.3
Execution Command:

```
yarn --cwd app/frontend playwright test testing/e2e/auth_rate_limiting_e2e.spec.ts
```

7.5 TC-NFR-SEC-04: Transport Encryption and Health Check Verification

Target Requirement: NFR-01.2, NFR-02.3

7.6 TC-NFR-SEC-05: RBAC and Live Map Integration Tests

Target Requirement: NFR-01.1, NFR-02.1

7.7 TC-NFR-SCAL-01: Scalability and Workload Capacity Load Test Evidence

Target Requirement: NFR-03.1, NFR-03.2
Execution Command:

```
locust -f locustfile.py --headless --users 1000 --spawn-rate 50 --run-time 3m \
  --host http://localhost:8000 --html report_1000_v2_2.html
```

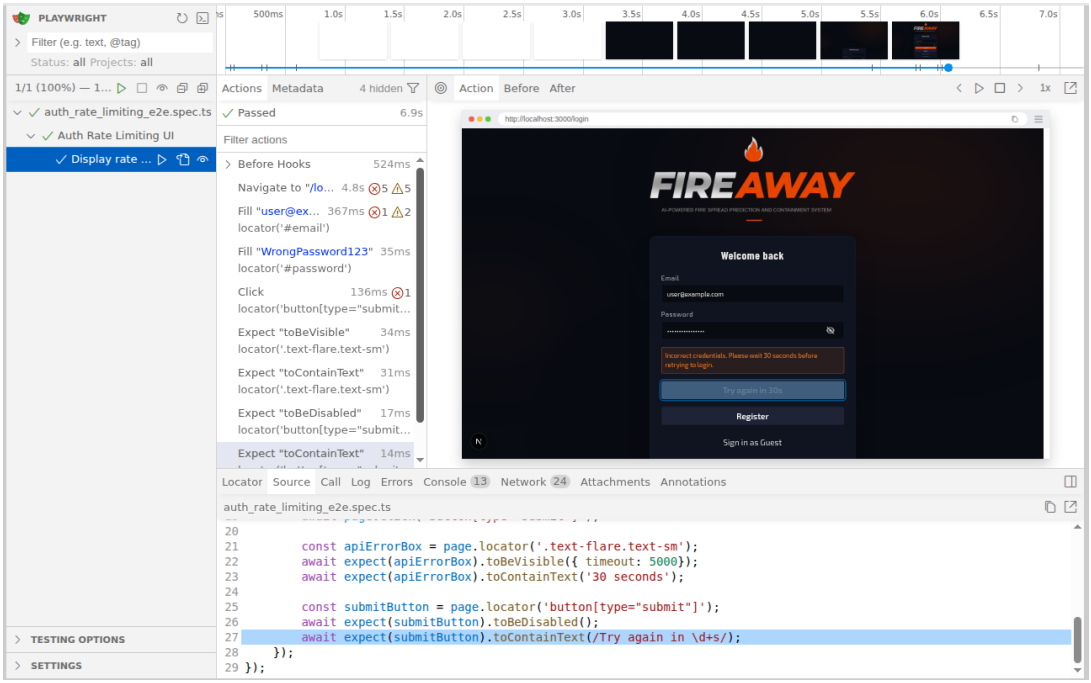


Figure 17: Playwright E2E assertion confirming UI cooldown timer and disabled submission button upon rate limit response.

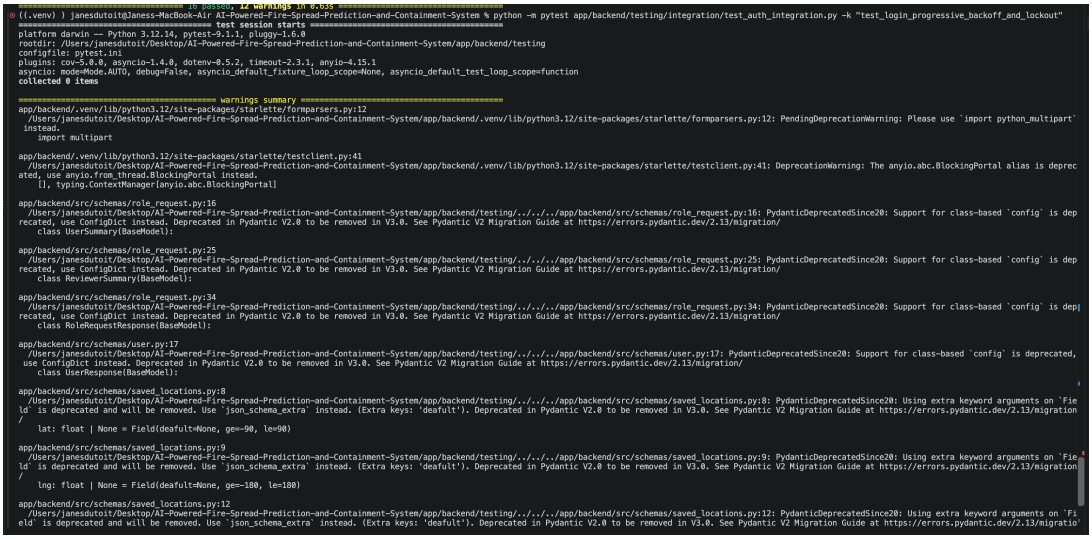


Figure 18: Progressive backoff validation under sequential failed login attempts (Stage 1).

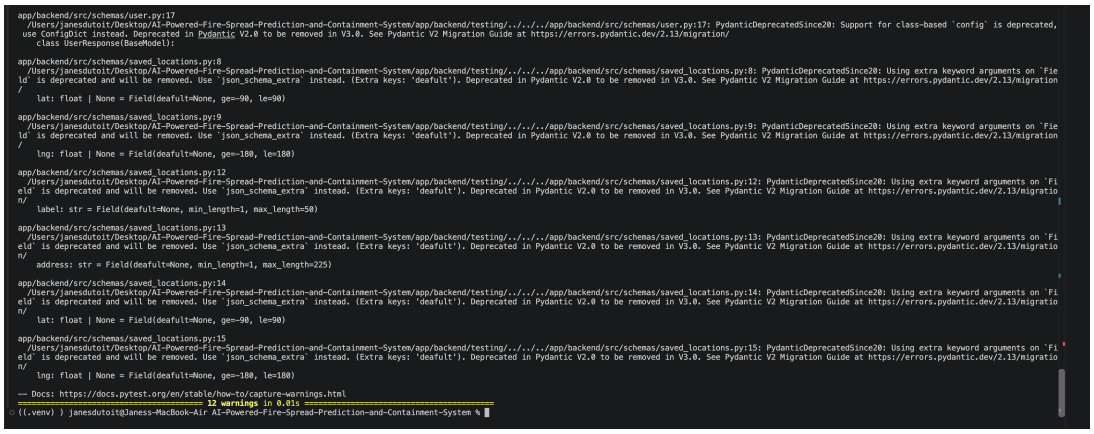


Figure 19: Progressive backoff validation confirming account lockout response (Stage 2).

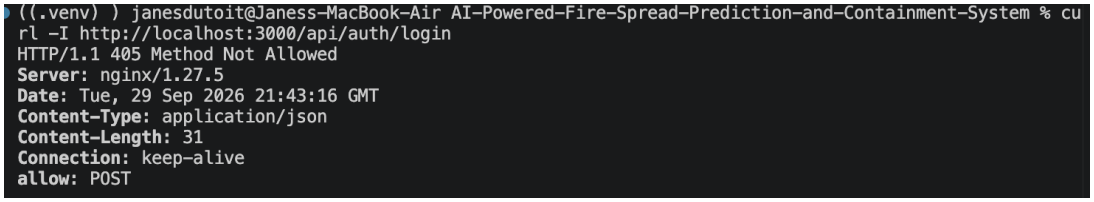


Figure 20: HTTP probe verifying redirect handling on authentication endpoints.

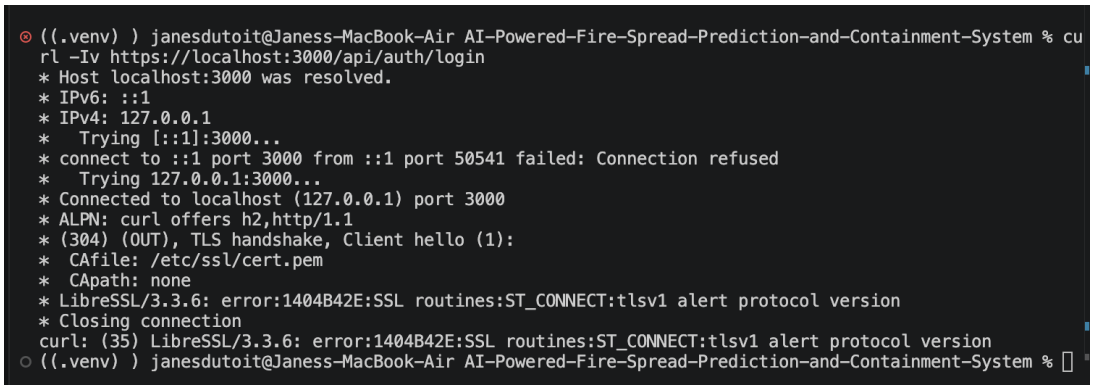


Figure 21: TLS 1.3 handshake and HTTPS transport security verification.

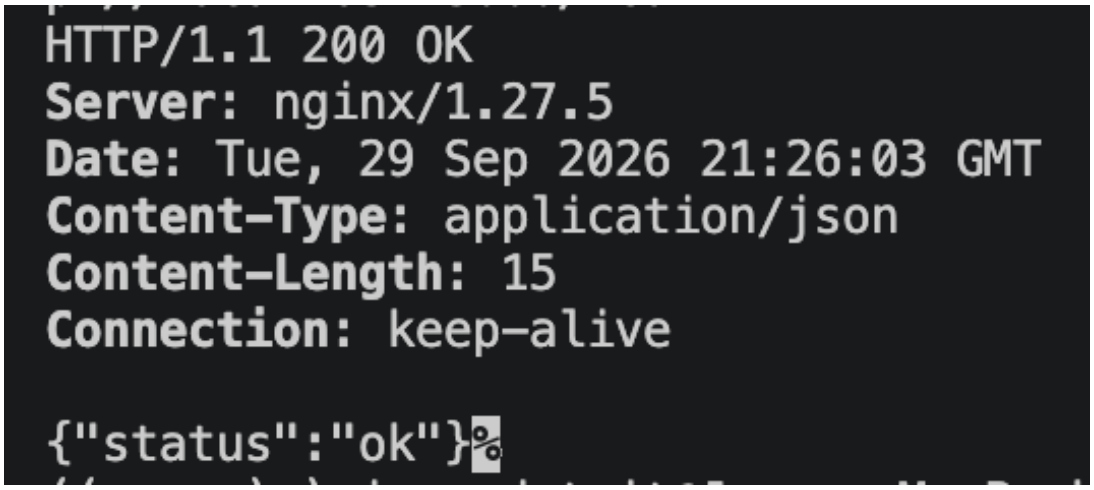


Figure 22: Liveness and system health check response latency verification.

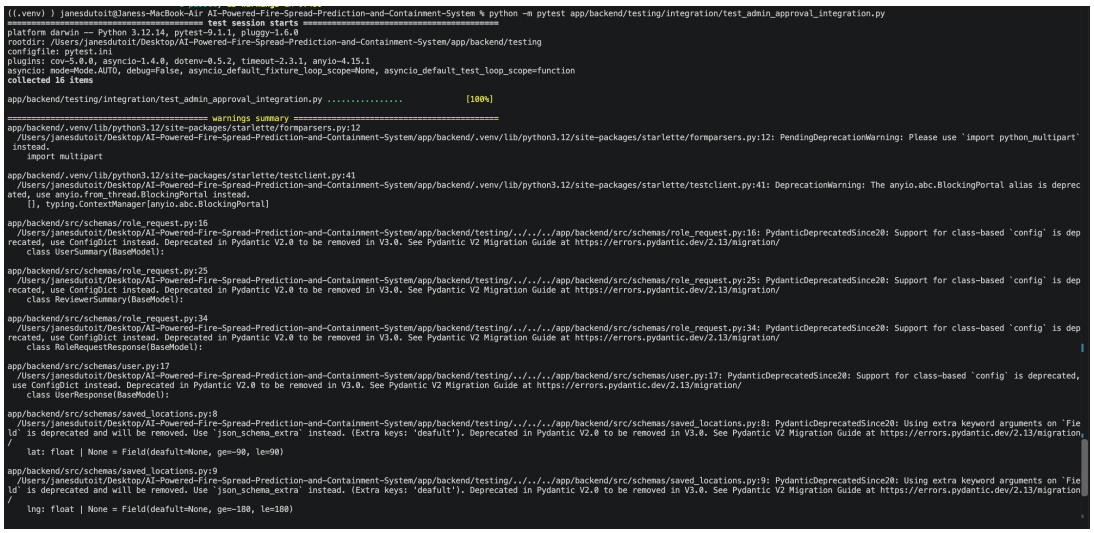


Figure 23: RBAC integration: Admin role review and approval verification (Part 1).

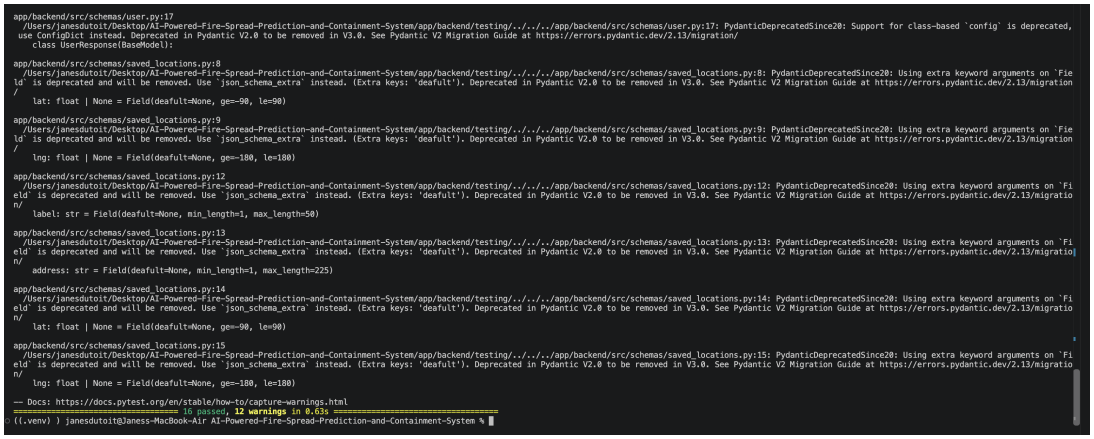


Figure 24: RBAC integration: Admin role rejection and restriction verification (Part 2).

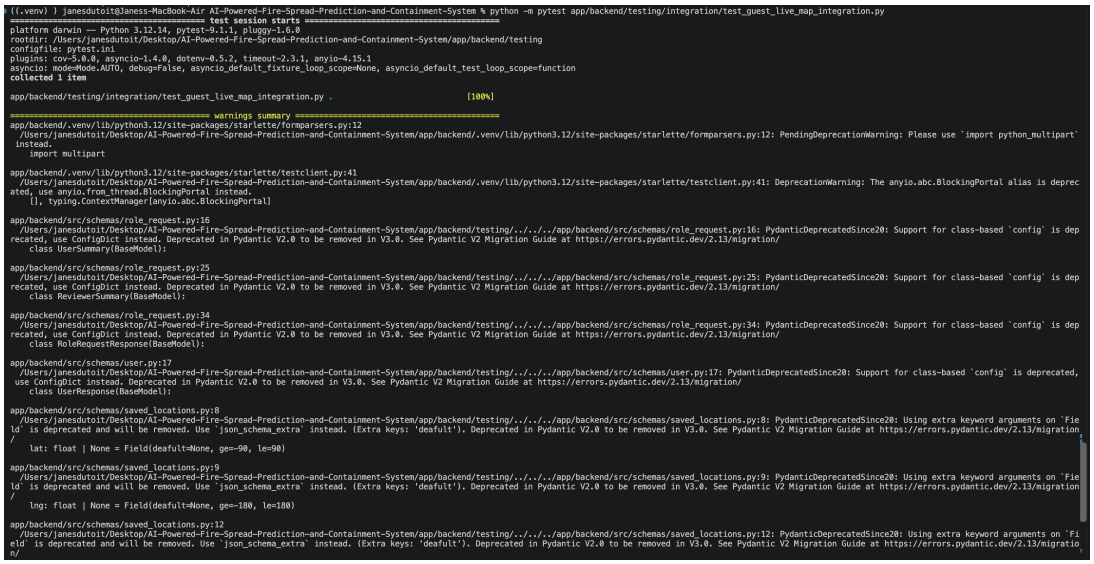


Figure 25: Guest map feed data consumption and marker display integration (Part 1).

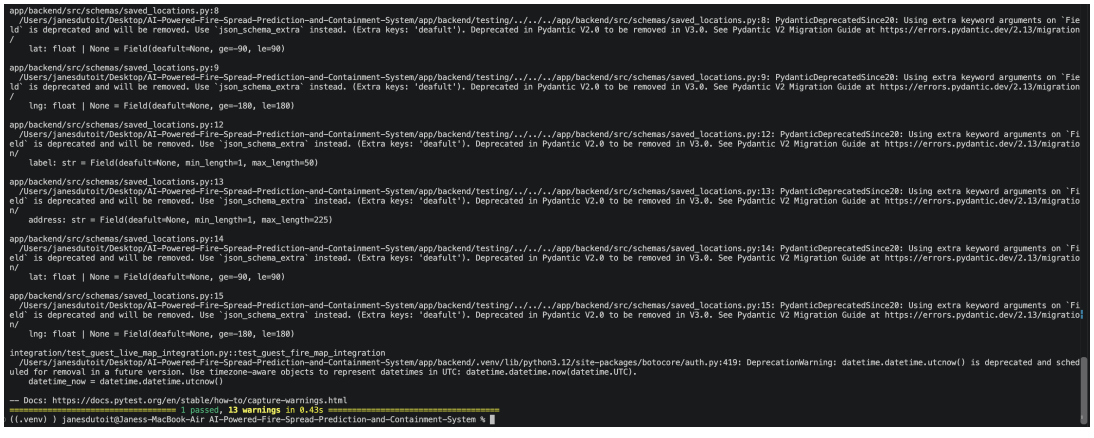


Figure 26: Guest map feed data consumption and marker display integration (Part 2).

7.7.1 Test Configuration

Load testing was executed using Locust across public ingestion and map query endpoints to verify scalability requirements NFR-03.1 and NFR-03.2:

- **Target Host:** Backend API direct interface (`http://localhost:8000`).
- **Concurrent Users:** 1,000 virtual users.
- **Spawn Rate:** 50 users per second.
- **Client Socket Timeout:** 5.0 seconds.
- **Task Distribution:** 75% public feed requests (GET `/api/guests/reported-fires`) and 25% report submissions (POST `/api/guests/reported-fires`).

7.7.2 Execution Results

Table 2 and Table 3 present the empirical metrics recorded by the test harness during the 1,000 concurrent user evaluation.

Endpoint	Requests	Failures	Failure %	Average (ms)	Median (ms)	p95 (ms)
GET `/api/guests/reported-fires`	3,798	3,796	99.95%	5,042.34	5,000	5,300
POST `/api/guests/reported-fires`	1,241	1,232	99.27%	5,024.91	5,000	5,300
Aggregated	**5,039**	**5,028**	**99.78%**	**5,038.01**	**5,000**	**5,300**

Table 2: Locust Load Test Performance Metrics Under 1,000 Concurrent Users

Endpoint	50%	75%	90%	95%	99%
GET `/api/guests/reported-fires`	5,000 ms	5,000 ms	5,100 ms	5,300 ms	5,300 ms
POST `/api/guests/reported-fires`	5,000 ms	5,000 ms	5,100 ms	5,300 ms	5,300 ms
Aggregated	**5,000 ms**	**5,000 ms**	**5,100 ms**	**5,300 ms**	**5,300 ms**

Table 3: Response Time Percentiles Distribution

7.7.3 Bottleneck Analysis and Verification Outcome

The system processed 5,039 requests with 5,028 failures, producing an aggregate failure rate of 99.78%. All recorded failures logged the exact client error `CatchResponseError('Failed with status: 0')`, indicating socket drops caused by the 5.0-second client timeout threshold rather than application validation errors.

The observed latency saturation (median response time of 5,000 ms and p95 of 5,300 ms) reveals that un-cached spatial database operations (ST_X, ST_Y) combined with worker process thread starvation exhaust the connection pool under high concurrency. Consequently, NFR-03.1 and NFR-03.2 did not meet the target pass criteria in this unoptimized run. Architectural mitigation requires scaling Uvicorn worker instances, tuning the PostgreSQL connection pool limit, and enabling Valkey response caching on read endpoints.

7.8 TC-NFR-ACC-01: Accessibility Compliance (WCAG 2.1 AA)

Target Requirement: NFR-04.1
Execution Command:

```
yarn --cwd app/frontend playwright test testing/e2e/accessibility.spec.ts
```

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	GET /api/guests/reported-fires	39530	32751	4283.53	13	5444	2573.72	131.9	109.28
POST	POST /api/guests/reported-fires	12958	10763	4297.11	13	5368	70.87	43.24	35.91
	Aggregated	52488	43514	4286.88	13	5444	1955.83	175.14	145.2

Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	GET /api/guests/reported-fires	5000	5000	5000	5000	5000	5100	5200	5400
POST	POST /api/guests/reported-fires	5000	5000	5000	5000	5000	5100	5200	5400
	Aggregated	5000	5000	5000	5000	5000	5100	5200	5400

Failures Statistics

# Failures	Method	Name	Message	First Seen	Last Seen
32751	GET	GET /api/guests/reported-fires	CatchResponseError('Failed with status: 0')	9/2/2026, 9:46:24 PM	9/2/2026, 9:50:57 PM
10763	POST	POST /api/guests/reported-fires	CatchResponseError('Failed with status: 0')	9/2/2026, 9:46:25 PM	9/2/2026, 9:50:57 PM

Figure 27: Locust HTML summary report documenting performance metrics and failure distribution under 1,000 concurrent virtual users.

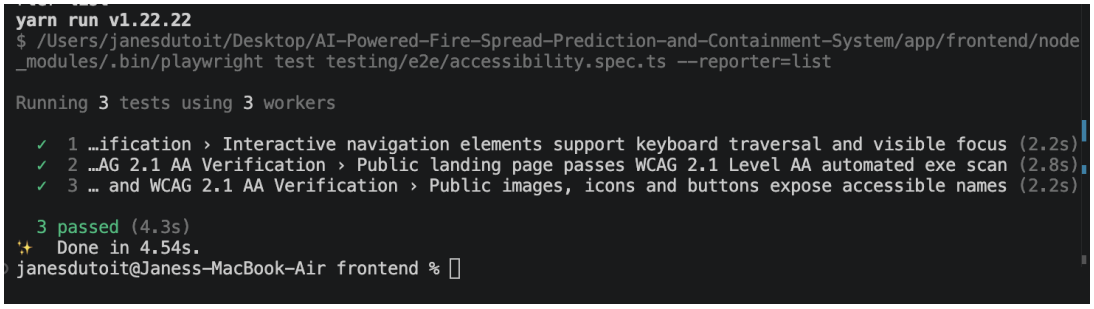


Figure 28: Playwright automated accessibility test suite summary execution results.

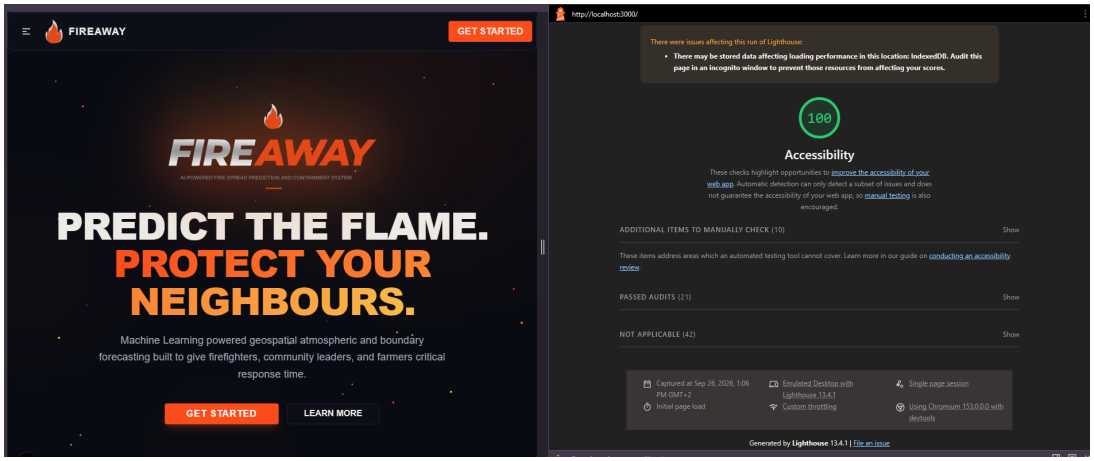


Figure 29: Accessibility evaluation: Public Landing Page

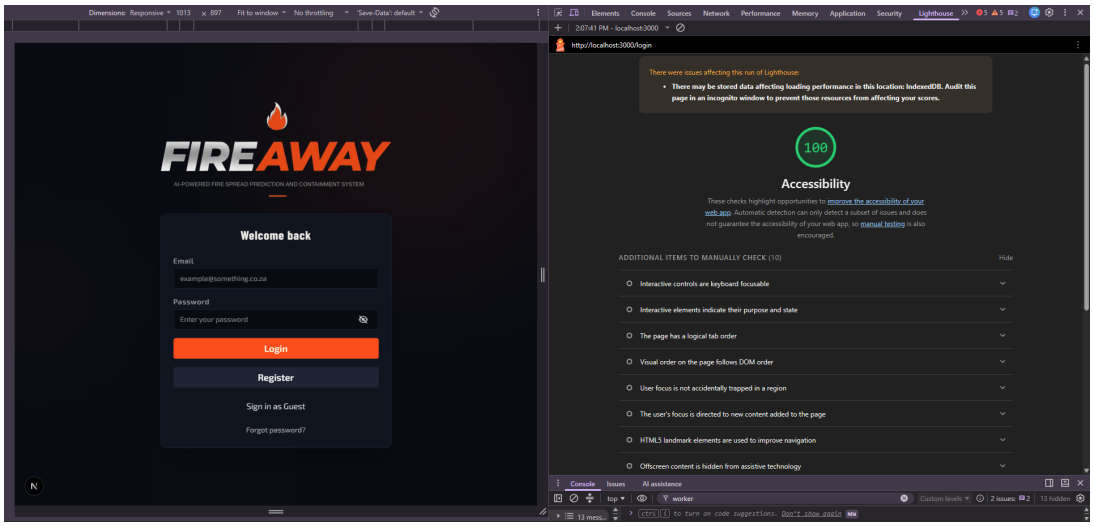


Figure 30: Accessibility evaluation: User Login View

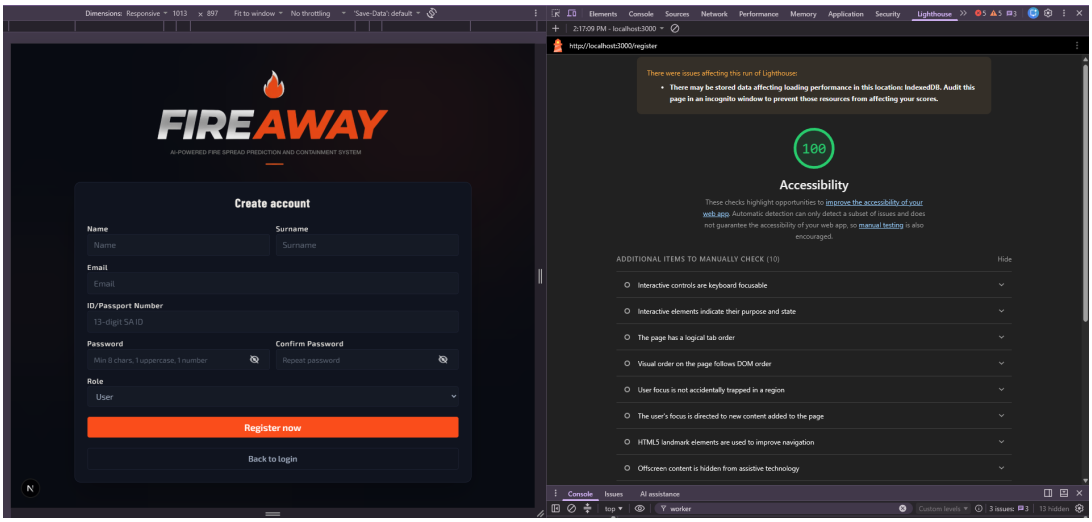


Figure 31: Accessibility evaluation: User Registration View

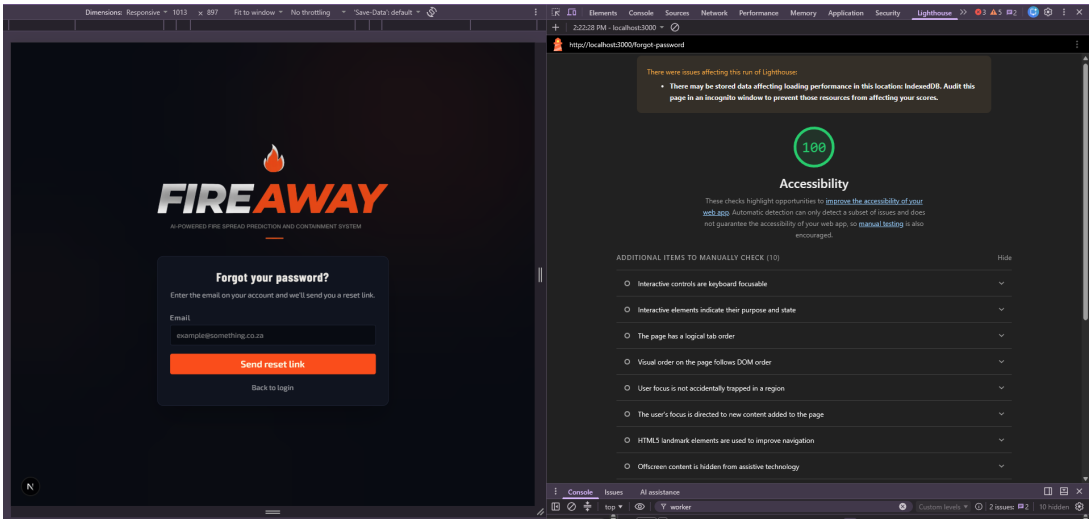


Figure 32: Accessibility evaluation: Forgot Password View

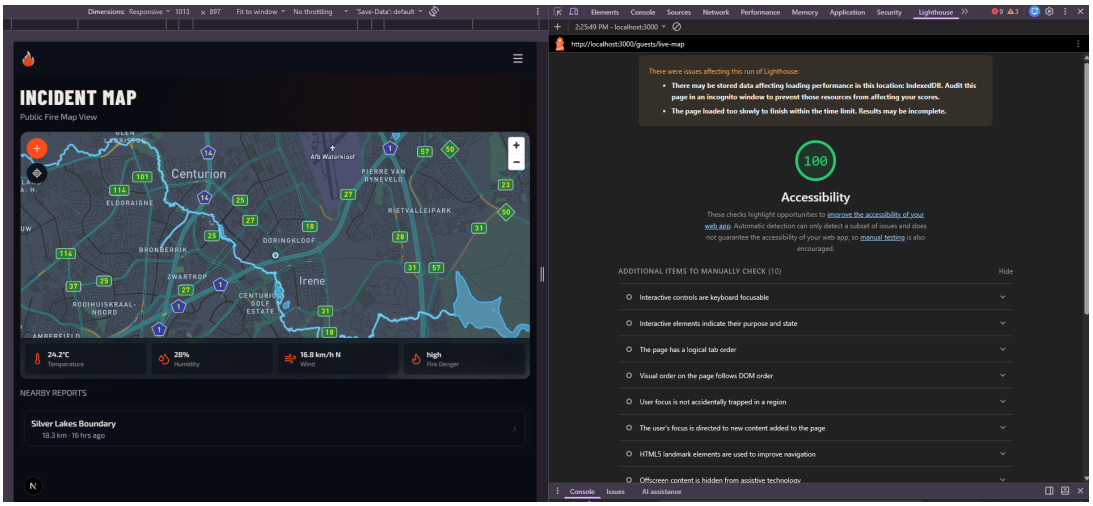


Figure 33: Accessibility evaluation: Live Fire Map View

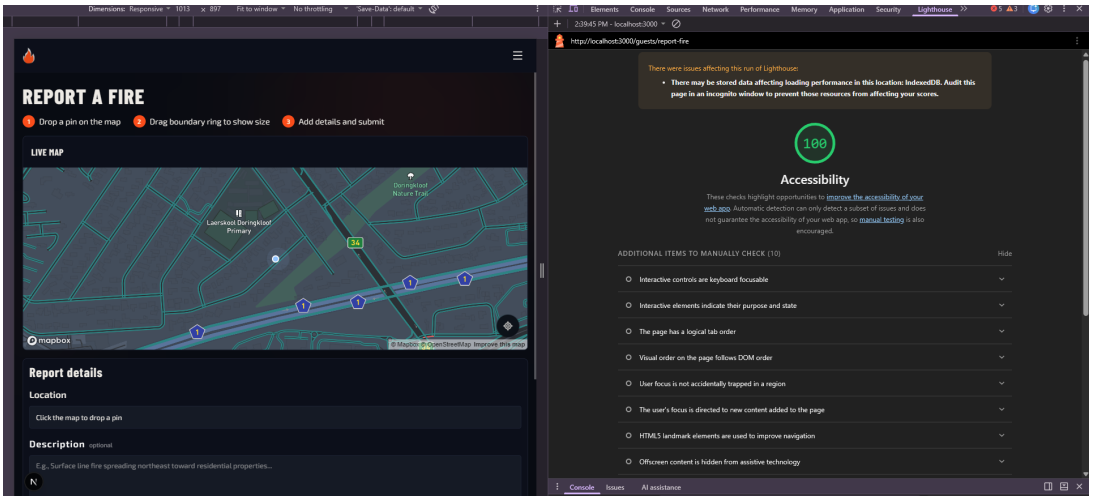


Figure 34: Accessibility evaluation: Report Fire View

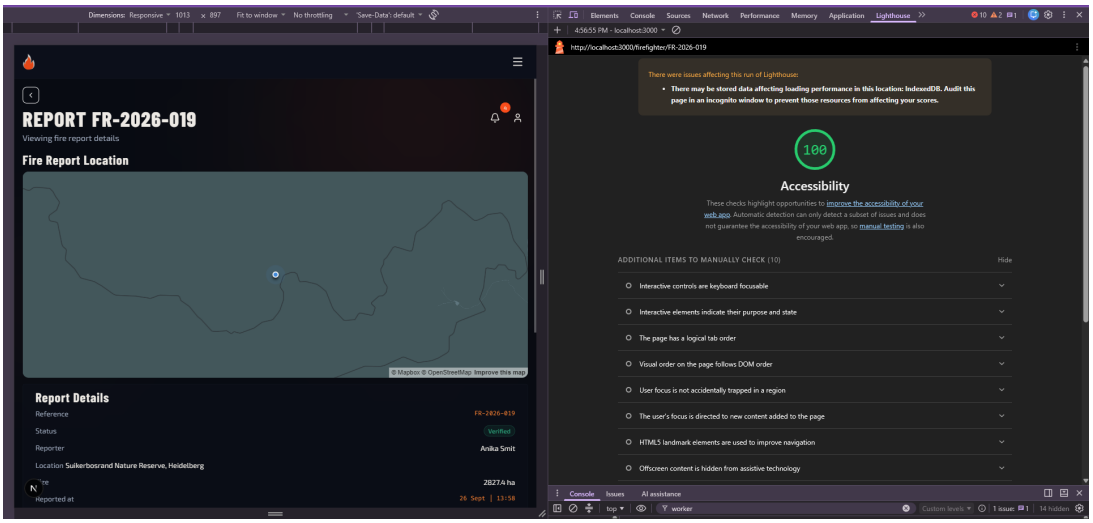


Figure 35: Accessibility evaluation: Fire Report View

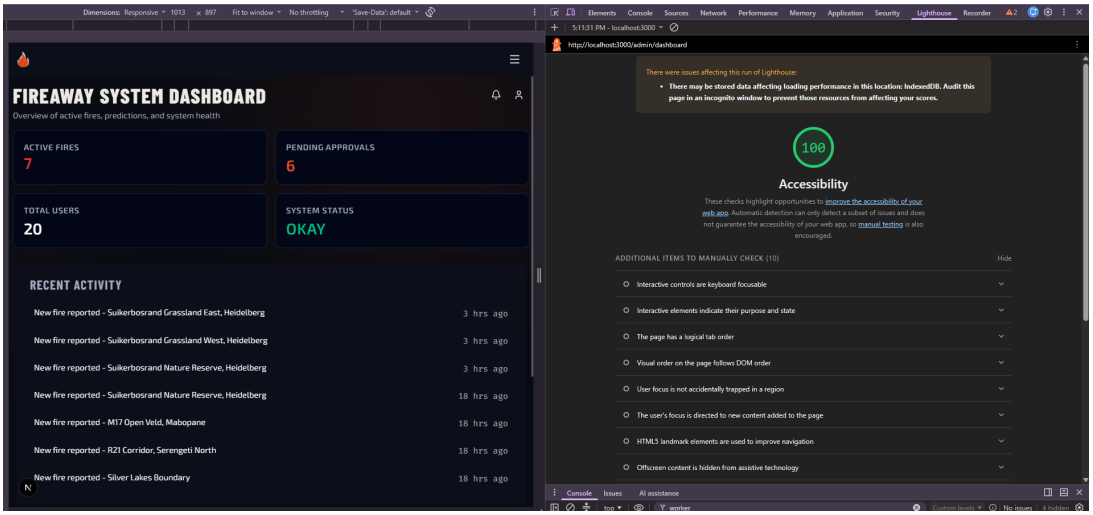


Figure 36: Accessibility evaluation: Administrator Dashboard

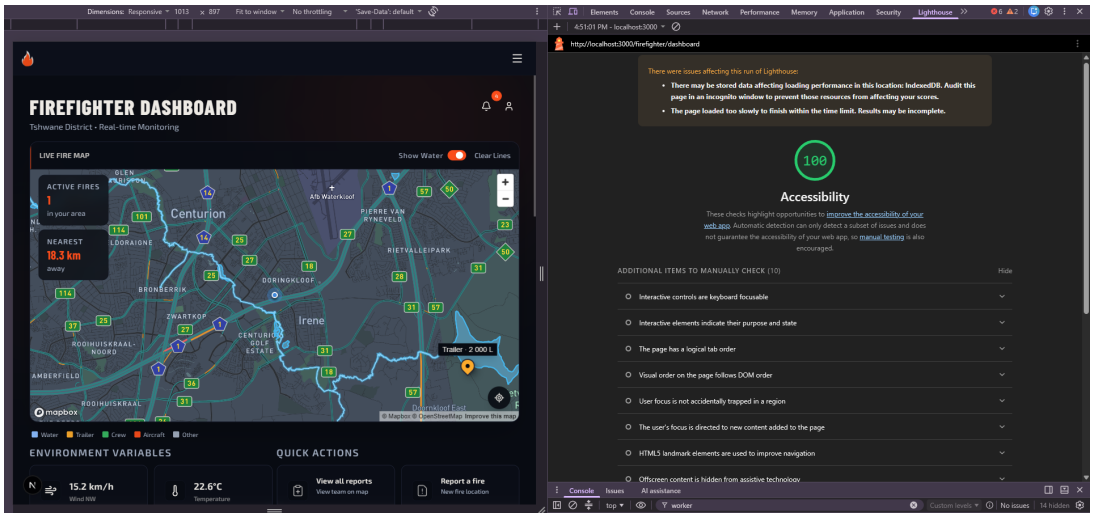


Figure 37: Accessibility evaluation: Firefighter Dashboard

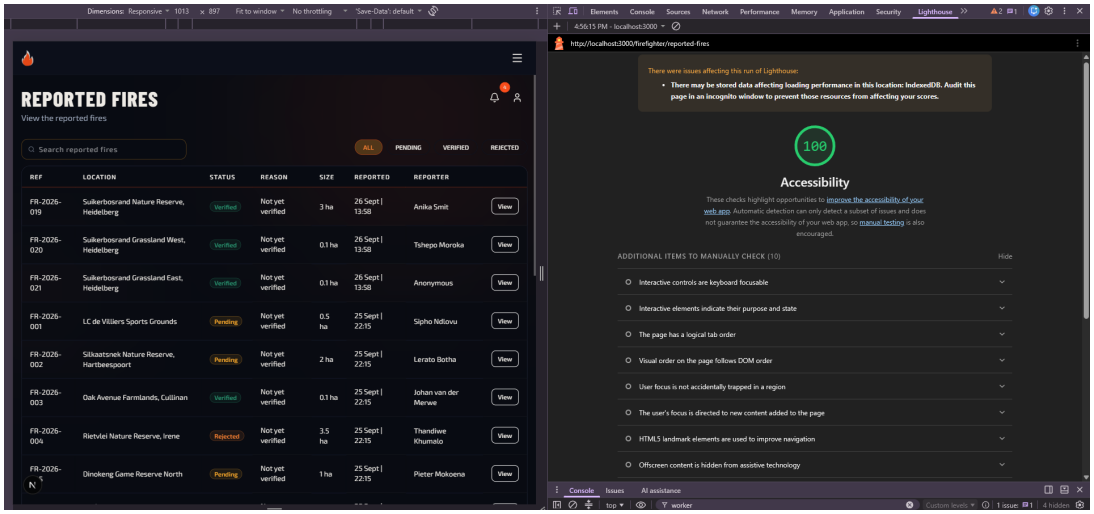


Figure 38: Accessibility evaluation: Firefighters Reported Fires Overview

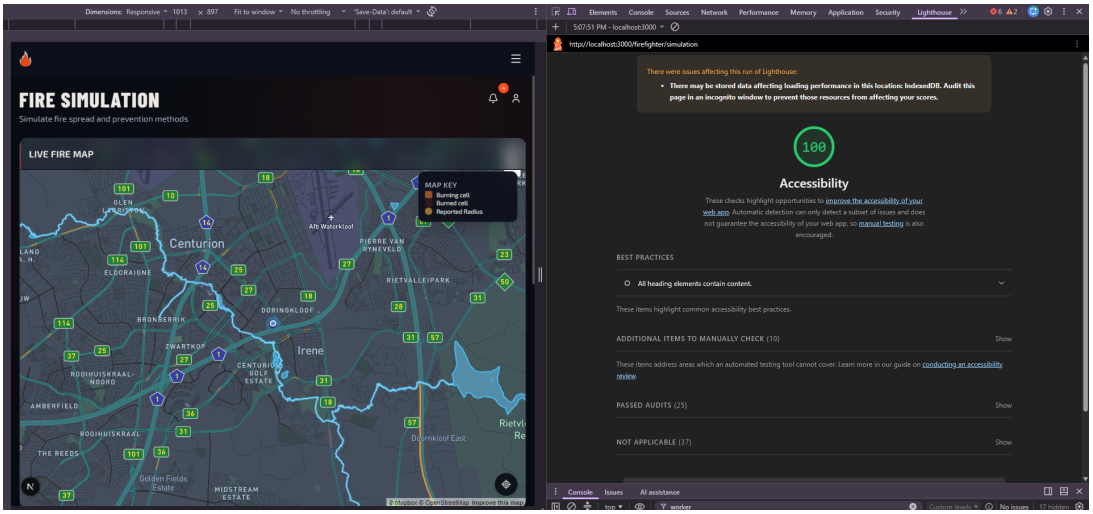


Figure 39: Accessibility evaluation: Firefighter Simulation View

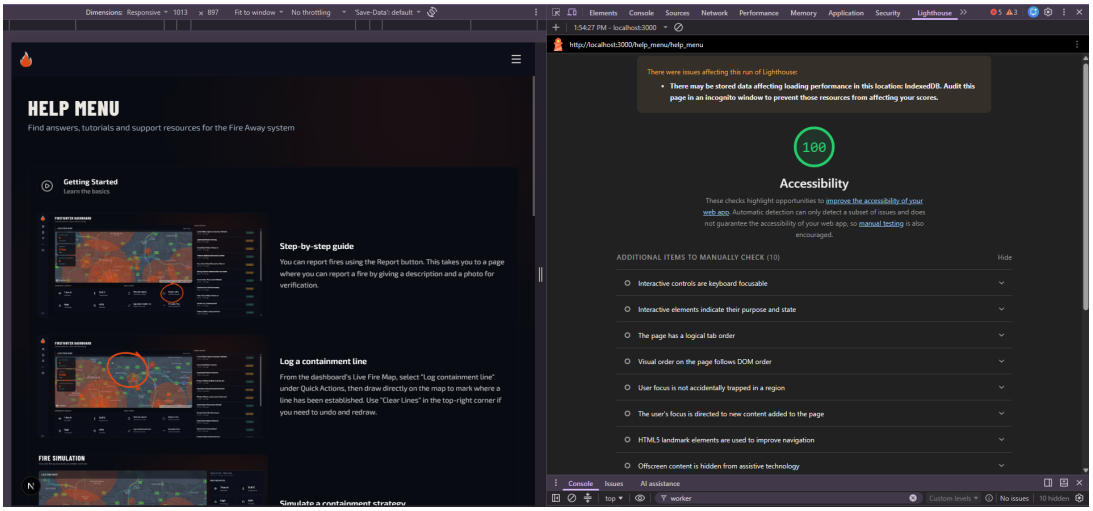


Figure 40: Accessibility evaluation: Help Menu View

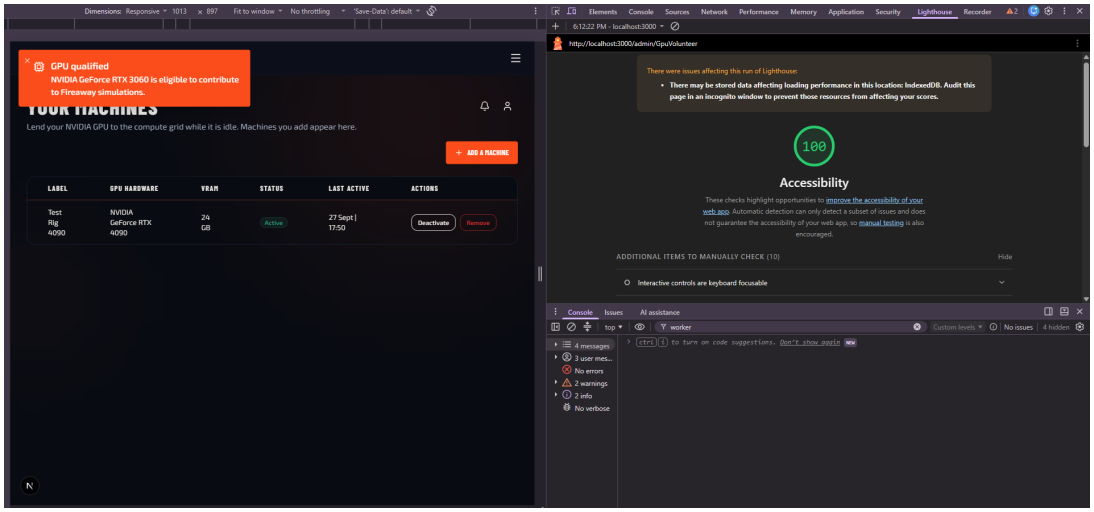


Figure 41: Accessibility evaluation: GPU Volunteer View

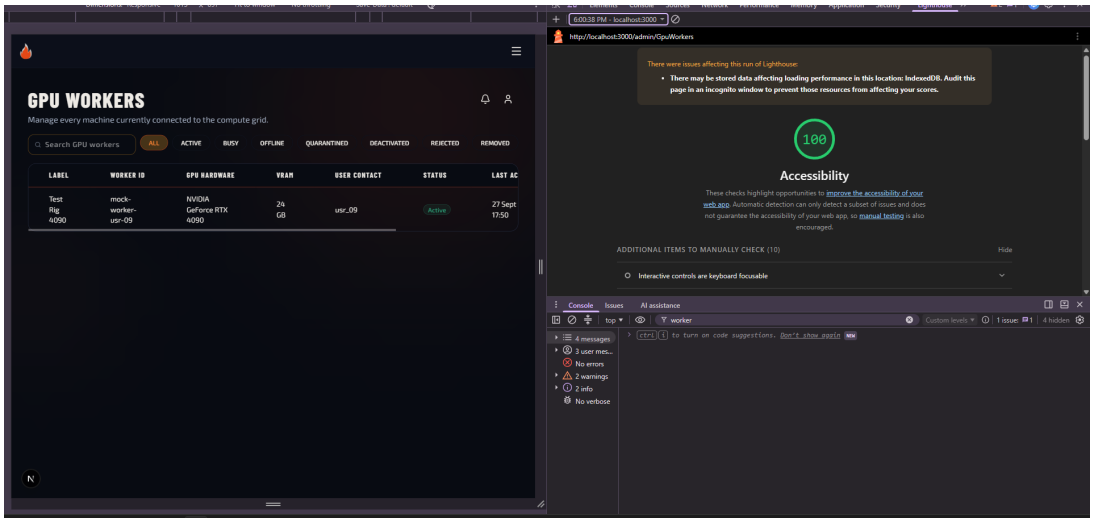


Figure 42: Accessibility evaluation: GPU Worker Dashboard

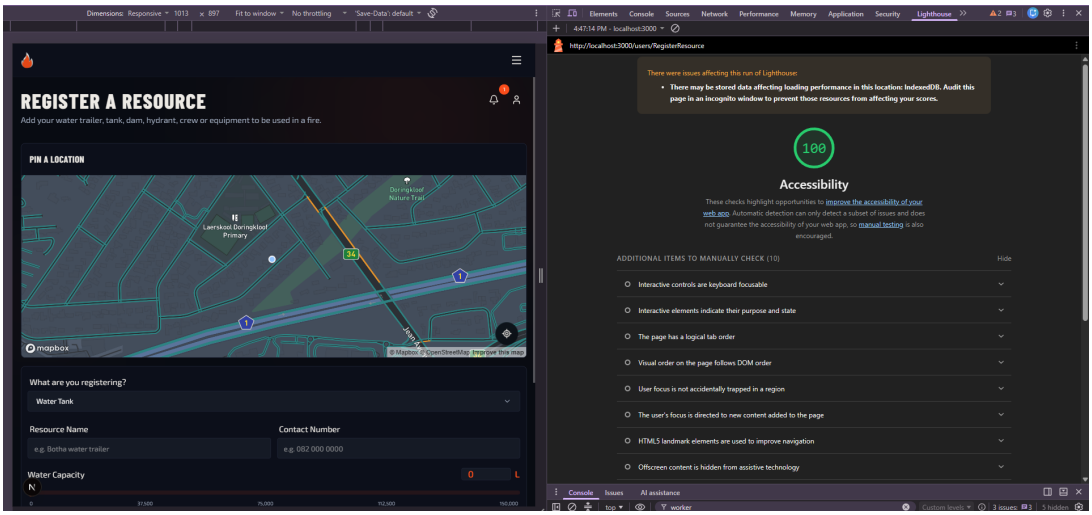


Figure 43: Accessibility evaluation: Register Resource View

8 Domain Model

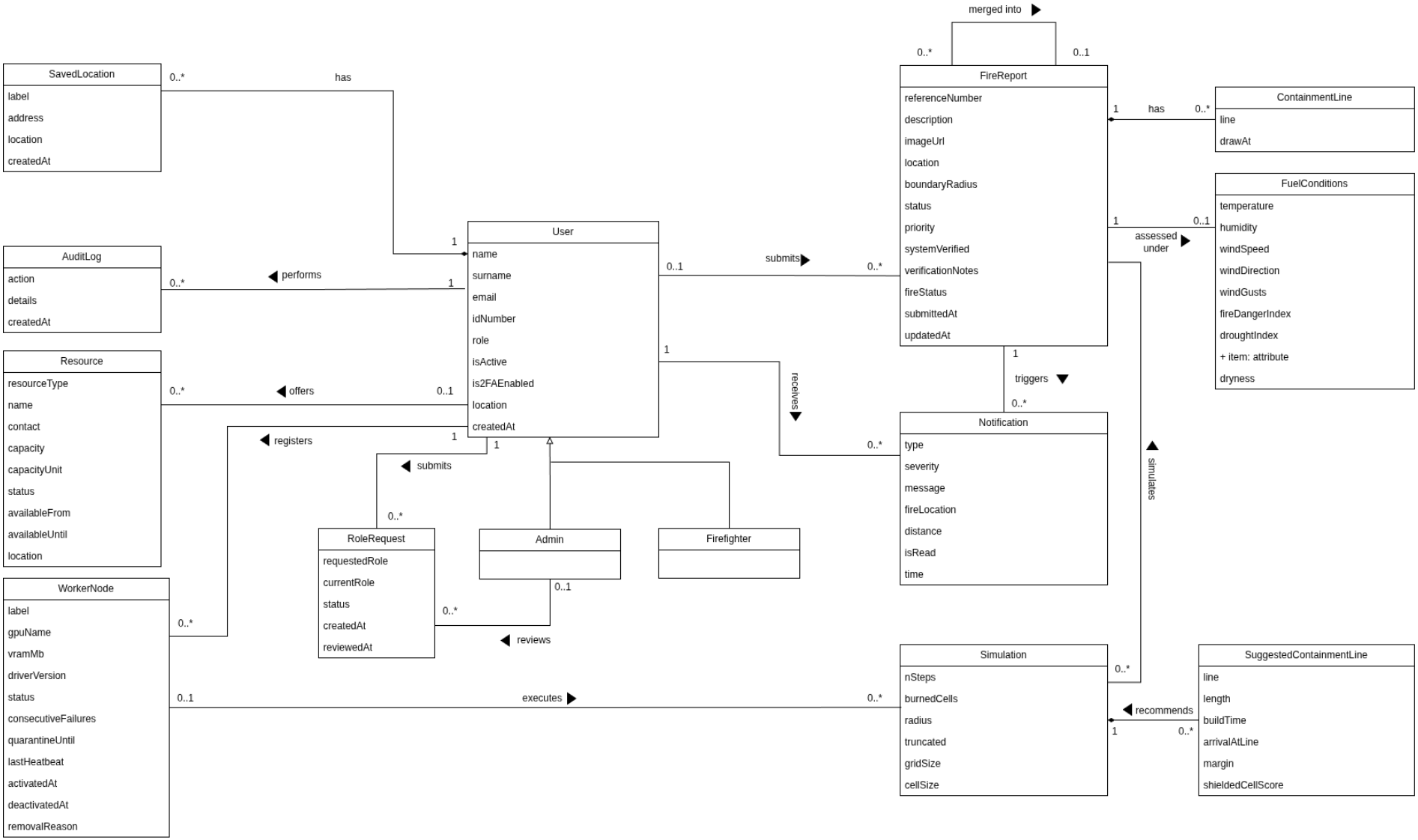


Figure 44: Domain Model

9 Change Log

Date	Version	Change
May 2026	1.0	Initial version
July 2026	2.0	Use Cases, NFR
August 2026	3.0	Use Cases, NFR
September 2026	4.0	Load Testing NFR Criteria, Accessibility Tests, and Complete Verification Evidence
October 2026	5.0	Domain Model